

Name: B SREE NIKHIL REDDY ID number: 2520090058

Week 4

SESSION-7

Task 1: Process Synchronization and Child Process Monitoring Using waitpid()

To create a child process, synchronize the parent with the child using waitpid(), monitor the child process, check normal termination, retrieve the exit status, handle signal termination, and test process synchronization.

CODE:

```
UN FICD 5.09 File: 22.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <signal.h>

int main() {
    pid_t pid;
    int status;

    // Create child process
    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        // Child process
        printf("Child Process: PID = %d\n", getpid());
        printf("Child is working...\n");
        sleep(2);
        printf("Child is terminating normally.\n");
        exit(0);
    }
    else {
        // Parent process
        printf("Parent Process: PID = %d\n", getpid());
        printf("Parent is waiting for child...\n");

        // Wait for specific child
        pid_t result = waitpid(pid, &status, 0);

        if (result == -1) {
            perror("waitpid failed");
            return 1;
        }

        printf("Parent: Child process completed.\n");

        // Check normal termination
        if (WIFEXITED(status)) {
            printf("Child terminated normally.\n");
            printf("Child exit status = %d\n", WEXITSTATUS(status));
        }

        // Check signal termination
        else if (WIFSIGNALED(status)) {
            printf("Child terminated by a signal.\n");
            printf("Signal number = %d\n", WTERMSIG(status));
        }
    }

    return 0;
}
```

OUTPUT:

```
Last login: Thu Aug 28 12:00:46 on console
screen@hillelreddy@Stress-MacBook-Pro ~ % nano 22.c
screen@hillelreddy@Stress-MacBook-Pro ~ % gcc 22.c -o 22
screen@hillelreddy@Stress-MacBook-Pro ~ % ./22
Parent Process: PID = 992
Parent is waiting for child...
Child Process: PID = 997
Child is working...
Child is terminating normally.
Parent: Child process completed.
Child terminated normally.
Child exit status = 28
screen@hillelreddy@Stress-MacBook-Pro ~ % █
```

Task 2: PATH Variable Retrieval and Command Resolution

To retrieve the PATH variable, parse its search directories, locate executable files, verify execute permissions, handle missing commands, and test command resolution.

CODE:

```
LOW WICD 5.09 File: 22.c

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

int main() {
    char *path;
    char *path_copy;
    char *dir;
    char full_path[5024];
    char command[100];

    // Get PATH variable
    path = getenv("PATH");

    if (path == NULL) {
        printf("PATH variable not found.\n");
        return 1;
    }

    printf("PATH = %s\n", path);

    // Get command from user
    printf("Enter command to search: ");
    scanf("%99s", command);

    // Copy PATH because strtok modifies the string
    path_copy = strdup(path);

    if (path_copy == NULL) {
        perror("strdup");
        return 1;
    }

    // Parse PATH directories
    dir = strtok(path_copy, ":");

    while (dir != NULL) {
        // Create complete path
        sprintf(full_path, "%s/%s", dir, command);

        // Check whether file exists and is executable
        if (access(full_path, X_OK) == 0) {
            printf("Command found!\n");
            printf("Executable path: %s\n", full_path);
            free(path_copy);
            return 0;
        }

        // Move to next directory
        dir = strtok(NULL, ":");
    }

    // Command not found
    printf("Command '%s' not found in PATH.\n", command);
    free(path_copy);
    return 0;
}
```

OUTPUT:

[illegible]

SESSION-8

Task 1: Variable Reference Detection and Value Expansion

To detect variable references, expand environment variable values, handle undefined variables, update tokens after expansion, support nested variables, and test expansion logic.

CODE:

```

1  UN PICO 5.09
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <string.h>
5  #include <ctype.h>
6
7  void expand_variables(char *input, char *output, int size) {
8      int i = 0, j = 0;
9
10     while (input[i] != '\0' && j < size - 1) {
11
12         // Detect $
13         if (input[i] == '$') {
14             i++;
15
16             // Read variable name
17             char var[500];
18             int k = 0;
19
20             while (isalnum(input[i]) || input[i] == '_') {
21                 if (k < 99)
22                     var[k++] = input[i];
23                 i++;
24             }
25
26             var[k] = '\0';
27
28             // Get environment variable value
29             char *value = getenv(var);
30
31             if (value != NULL) {
32                 // Expand variable value
33                 for (int x = 0; value[x] != '\0' && j < size - 1; x++) {
34                     output[j++] = value[x];
35                 }
36             } else {
37                 // Undefined variable -> empty string
38                 printf("Undefined variable: %s\n", var);
39             }
40             // else {
41             //     output[j++] = input[i++];
42             // }
43         }
44         output[j] = '\0';
45     }
46 }
47
48 int main() {
49     char input[500];
50     char output[1000];
51
52     printf("Enter text with variables: ");
53     fgets(input, sizeof(input), stdin);
54
55     // Remove newline
56     input[strlen(input, '\n')] = '\0';
57
58     printf("\nOriginal input: %s\n", input);
59
60     expand_variables(input, output, sizeof(output));
61
62     printf("Expanded input: %s\n", output);
63
64     return 0;
65 }

```

OUTPUT:

```
screenikhlreddy@Stress-MacBook-Pro ~ % nano 24.c
screenikhlreddy@Stress-MacBook-Pro ~ % gcc 24.c -o 24
screenikhlreddy@Stress-MacBook-Pro ~ % ./24
Enter text with variables: 12

Original input: 12
Expanded input: 12
screenikhlreddy@Stress-MacBook-Pro ~ % nano 24.c
screenikhlreddy@Stress-MacBook-Pro ~ %
```

Task 2: Built-in Identification, Dispatch Table and In-Process Execution

To identify built-in commands, create a dispatch table, execute in-process commands, handle invalid commands, maintain shell state, and test dispatch logic.

CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define MAX_INPUT 100

// Function declarations
void shell_cd(char *args[]);
void shell_pwd(char *args[]);
void shell_help(char *args[]);
void shell_exit(char *args[]);

// Dispatch table structure
struct builtin {
    char *name;
    void (*function)(char *args[]);
};

// Built-in command table
struct builtin builtins[] = {
    {"cd", shell_cd},
    {"pwd", shell_pwd},
    {"help", shell_help},
    {"exit", shell_exit}
};

int builtin_count = 4;

// cd command
void shell_cd(char *args[]) {
    if (args[1] == NULL) {
        printf("cd: missing directory\n");
        return;
    }

    if (chdir(args[1]) != 0) {
        perror("cd");
    }
}

// pwd command
void shell_pwd(char *args[]) {
    char cwd[1024];

    if (getcwd(cwd, sizeof(cwd)) != NULL) {
        printf("%s\n", cwd);
    } else {
        perror("pwd");
    }
}

// help command
void shell_help(char *args[]) {
    printf("Built-in commands:\n");
    printf("  cd <directory>    - Change directory\n");
    printf("  pwd              - Show current directory\n");
    printf("  help             - Show help\n");
    printf("  exit             - Exit shell\n");
}

// exit command
void shell_exit(char *args[]) {
    printf("Exiting shell...\n");
    exit(0);
}

// Identify and execute built-in command
int execute_builtin(char *args[]) {
    for (int i = 0; i < builtin_count; i++) {
        if (strcmp(args[0], builtins[i].name) == 0) {
            builtins[i].function(args);
            return 0;
        }
    }
    return 1;
}
```

OUTPUT:

```
Last login: Thu Aug 28 12:14:45 on tty000
screenkhilreddy@Srees-MacBook-Pro ~ % nano 25.c

screenkhilreddy@Srees-MacBook-Pro ~ % gcc 25.c -o
clang: error: argument to '-o' is missing (expected 1 value)
screenkhilreddy@Srees-MacBook-Pro ~ % gcc 25.c -o 25
screenkhilreddy@Srees-MacBook-Pro ~ % ./25
Simple Shell
Type 'help' for commands.
myshell> exit
Exiting shell....
screenkhilreddy@Srees-MacBook-Pro ~ % ./25
Simple Shell
Type 'help' for commands.
myshell> pwd
Command not found: pwd
myshell> cd
/Users/screenkhilreddy
myshell> █
```